

SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CSA1735 – ARTIFICIAL INTELLIGENCE

SLOT A

LAB EXPERIMENTS

THIRULOGASUNDAR - 192321073

THIRULOGASUNDAR Y - 192321073

FACULTY

Dr. Thalapathi Rajasekaran R

Experiment 1- 8 PUZZLE

Aim

To write a Python program to solve the 8-Puzzle problem using the A* search algorithm.

Algorithm

1. Start with the initial state of the puzzle and the goal state.
2. Represent the empty tile as 0.
3. Calculate the heuristic value using the Manhattan distance of each tile from its goal position.
4. Calculate the evaluation function \(f(n)=g(n)+h(n)\), where \(g(n)\) is the path cost and \(h(n)\) is the heuristic cost.
5. Select the state with the lowest evaluation value.
6. Generate possible states by moving the empty tile up, down, left, or right.
7. Repeat until the goal state is reached.
8. Display the sequence of puzzle states.

THIRULOGASUNDAR - 192321073

Program

```
import heapq

start = (1, 2, 3, 4, 0, 6, 7, 5, 8)
goal = (1, 2, 3, 4, 5, 6, 7, 8, 0)

def heuristic(state):
    distance = 0
    for i in range(9):
        if state[i] != 0:
```

```
            j = goal.index(state[i])
            distance += abs(i // 3 - j // 3)
            distance += abs(i % 3 - j % 3)
    return distance

def solve():
    queue = [(heuristic(start), 0, start, [])]
    visited = set()

    while queue:
        f, cost, state, path = heapq.heappop(queue)

        if state in visited:
            continue
        visited.add(state)
        path = path + [state]

        if state == goal:
            for s in path:
                for i in range(0, 9, 3):
                    print(s[i:i+3])
                print()
            return

        zero = state.index(0)
        r, c = zero // 3, zero % 3
```

THIRULOGASUNDAR - 192321073

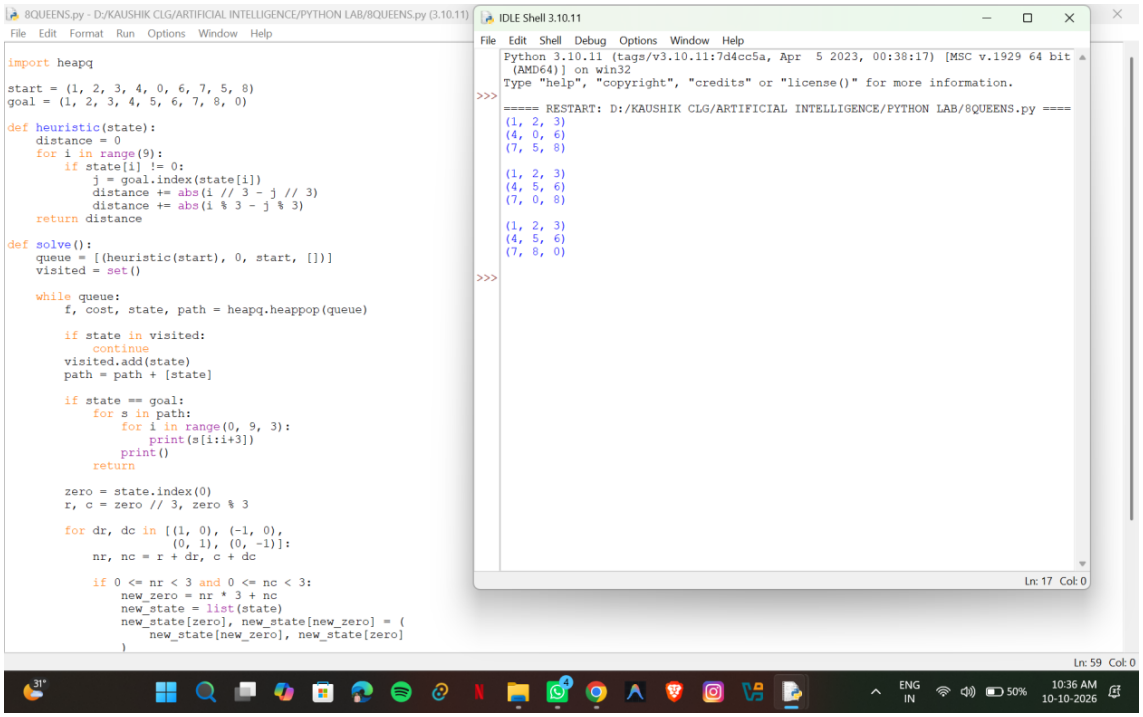
```
        for dr, dc in [(1, 0), (-1, 0),
                       (0, 1), (0, -1)]:
            nr, nc = r + dr, c + dc

            if 0 <= nr < 3 and 0 <= nc < 3:
                new_zero = nr * 3 + nc
                new_state = list(state)
                new_state[zero], new_state[new_zero] = (
                    new_state[new_zero], new_state[zero]
                )
                new_state = tuple(new_state)
```

THIRULOGASUNDAR - 192321073

```
                if new_state not in visited:
                    new_cost = cost + 1
                    heapq.heappush(
                        queue,
                        (new_cost + heuristic(new_state),
                         new_cost, new_state, path)
                    )
solve()
```

Output



Result: The 8-Puzzle problem is solved successfully using the A* search algorithm.

Experiment 2: 8-Queen Problem

Aim

To write a Python program to solve the 8-Queen problem using the backtracking technique.

Algorithm

1. Start with an empty chessboard.
2. Place a queen in the first row.
3. Check whether the selected column is safe from other queens.
4. Ensure that no two queens share the same column or diagonal.
5. If the position is safe, place the queen and move to the next row.
6. If no safe position exists, backtrack to the previous row.
7. Repeat until eight queens are placed successfully.
8. Display the chessboard.

THIRULOGASUNDAR - 192321073

Program

```
N = 8
board = [-1] * N


def safe(row, col):
    for r in range(row):
        if board[r] == col:
            return False
        if abs(board[r] - col) == abs(r - row):
            return False
    return True
```

```
def solve(row):
    if row == N:
        return True

    for col in range(N):
        if safe(row, col):
            board[row] = col

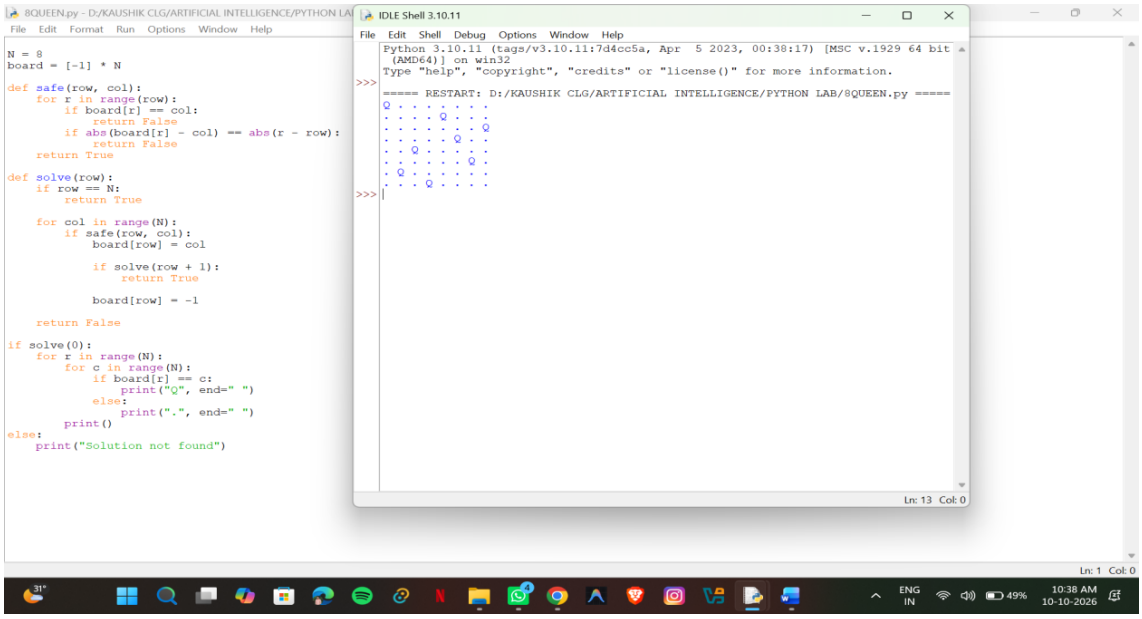
            if solve(row + 1):
                return True

            board[row] = -1
    return False

if solve(0):
    for r in range(N):
        for c in range(N):
            if board[r] == c:
                print("Q", end=" ")
            else:
                print(".", end=" ")
        print()
else:
    print("Solution not found")
```

THIRULOGASUNDAR - 192321073

Output



Result: The 8-Queen problem is solved successfully using backtracking.

THIRULOGASUNDAR - 192321073

Experiment 3: Water Jug Problem

Aim

To write a Python program to solve the Water Jug problem using the Breadth-First Search (BFS) algorithm.

Algorithm

1. Initialize two water jugs with capacities of 4 litres and 3 litres.
2. Set the initial state as (0, 0).
3. Define the goal as obtaining 2 litres of water in either jug.
4. Generate possible states by filling a jug, emptying a jug, or pouring water from one jug into the other.
5. Add each unvisited state to a queue.
6. Repeat until the goal state is reached.
7. Display the sequence of states.

Program

THIRULOGASUNDAR - 192321073

```
from collections import deque


def water_jug():
    capacity1 = 4
    capacity2 = 3
    start = (0, 0)


    queue = deque([(start, [start])])
    visited = {start}


    while queue:
        (a, b), path = queue.popleft()
```

```
        if a == 2 or b == 2:
            print("Solution:")
            for state in path:
                print(state)
            return

        amount = min(a, capacity2 - b)
        amount2 = min(b, capacity1 - a)

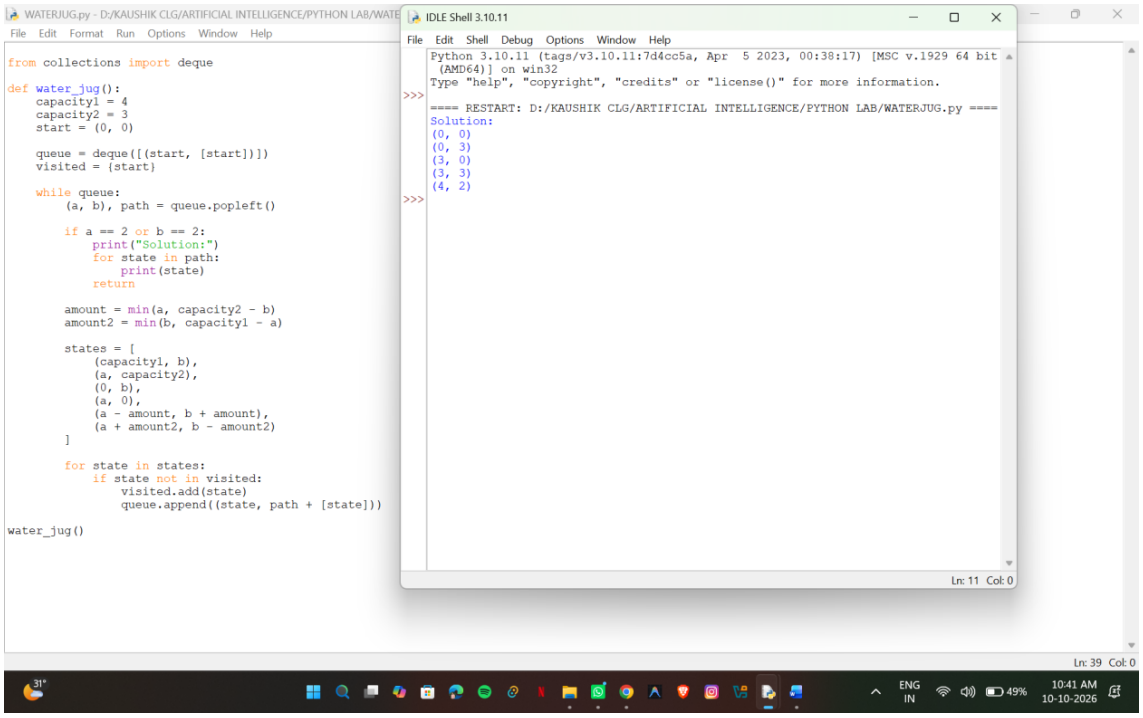
        states = [
            (capacity1, b),
            (a, capacity2),
            (0, b),
            (a, 0),
            (a - amount, b + amount),
            (a + amount2, b - amount2)
        ]

        for state in states:
            if state not in visited:
                visited.add(state)
                queue.append((state, path + [state]))

water_jug()
```

THIRULOGASUNDAR - 192321073

Output



Result: The Water Jug problem is solved successfully using BFS to obtain 2 litres of water.

THIRULOGASUNDAR - 192321073

Experiment 4: Crypt-Arithmetic Problem

Aim

To write a Python program to solve the crypt-arithmetic problem SEND + MORE = MONEY using backtracking and digit assignment.

Algorithm

1. Identify the unique letters in the words SEND, MORE, and MONEY.
2. Assign a different digit from 0 to 9 to each letter.
3. Ensure that the first letters S and M are not zero.
4. Convert the words into their corresponding numbers.
5. Check whether the equation SEND + MORE = MONEY is satisfied.
6. If the equation is true, display the digit assignments and the arithmetic result.
7. Otherwise, continue searching for another assignment.

THIRULOGASUNDAR - 192321073

Program

```
from itertools import permutations

letters = "SENDMORY"

for p in permutations(range(10), len(letters)):
    d = dict(zip(letters, p))

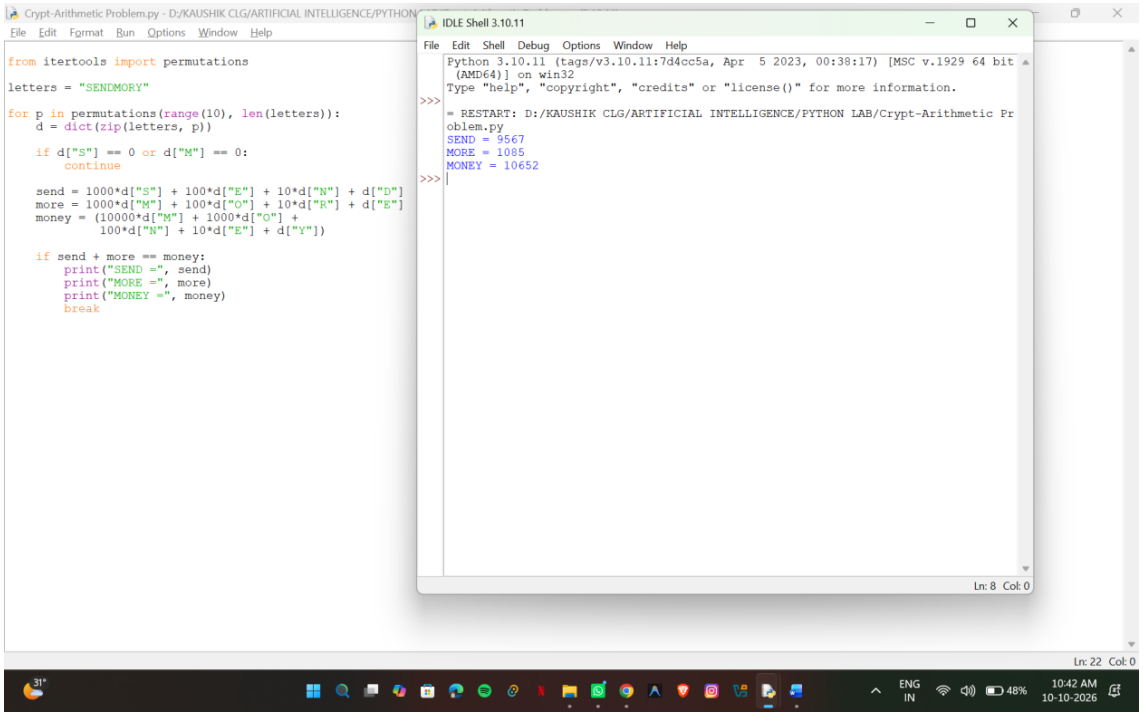
    if d["S"] == 0 or d["M"] == 0:
        continue

    send = 1000*d["S"] + 100*d["E"] + 10*d["N"] + d["D"]
    more = 1000*d["M"] + 100*d["O"] + 10*d["R"] + d["E"]
```

```
    money = (10000*d["M"] + 1000*d["O"] +
             100*d["N"] + 10*d["E"] + d["Y"])

    if send + more == money:
        print("SEND =", send)
        print("MORE =", more)
        print("MONEY =", money)
        break
```

Output



Result: The crypt-arithmetic problem is solved successfully by assigning unique digits to the letters.

Experiment 5: Missionaries and Cannibals Problem

Aim

To write a Python program to solve the Missionaries and Cannibals problem using the Breadth-First Search (BFS) algorithm.

Algorithm

1. Initialize three missionaries and three cannibals on the left bank of the river.
2. Place the boat on the left bank with a capacity of two people.
3. Generate possible boat movements carrying one or two people.
4. Ensure that cannibals never outnumber missionaries on either bank when missionaries are present.
5. Use BFS to explore all valid states without revisiting previously explored states.
6. Continue until all missionaries and cannibals reach the right bank.
7. Display the sequence of valid states.

THIRULOGASUNDAR - 192321073

Program

```
from collections import deque


start = (3, 3, 1)
goal = (0, 0, 0)


moves = [(1, 0), (2, 0), (0, 1),
         (0, 2), (1, 1)]


def valid(m, c):
    mr = 3 - m
    cr = 3 - c
```

```
    return (0 <= m <= 3 and 0 <= c <= 3
            and (m == 0 or m >= c)
            and (mr == 0 or mr >= cr))

def solve():
    queue = deque([(start, [start])])
    visited = {start}

    while queue:
        state, path = queue.popleft()
        m, c, boat = state
        if state == goal:
            for s in path:
                print(s)
            return

        for dm, dc in moves:
            if boat == 1:
                new_state = (m - dm, c - dc, 0)
            else:
                new_state = (m + dm, c + dc, 1)

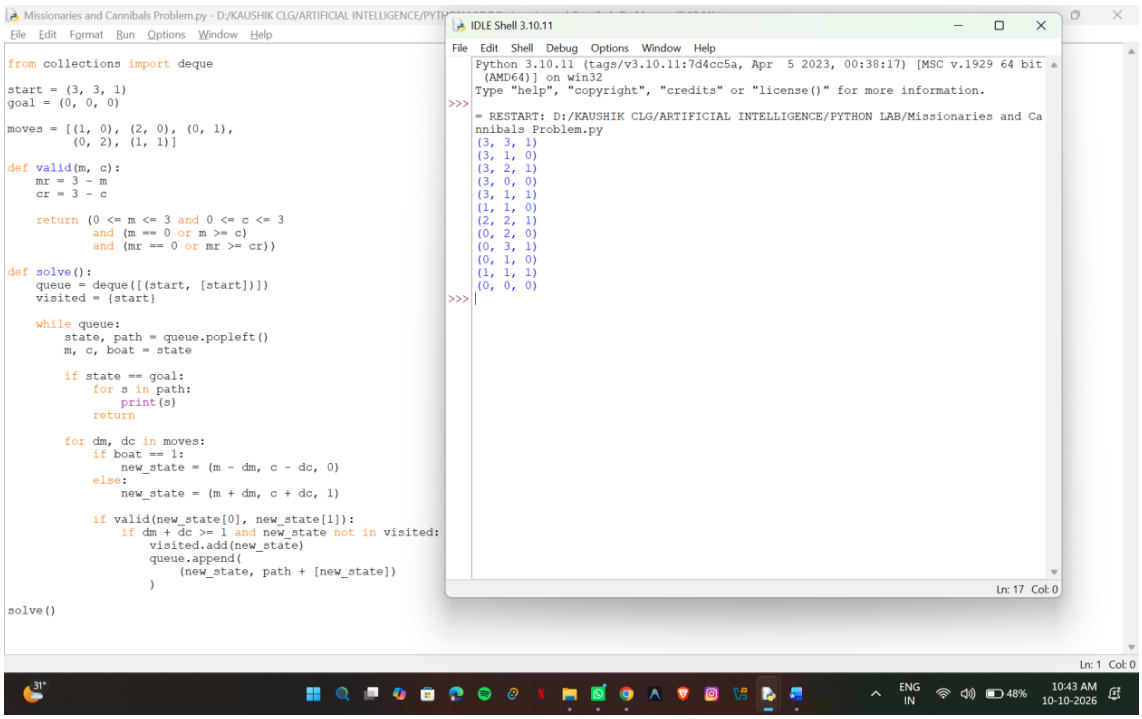
            if valid(new_state[0], new_state[1]):
                if dm + dc >= 1 and new_state not in visited:
```

THIRULOGASUNDAR - 192321073

```
            visited.add(new_state)
            queue.append(
                (new_state, path + [new_state])
            )
```

solve()

Output



Result: The Missionaries and Cannibals problem is solved successfully using BFS while satisfying the safety constraints.